

Assignment 1 : MA20268

Disclaimer: This project report is made (typed & thought upon) completely by hand and with full understanding of the content by me

– Shubham Krishan, 24MA10063

Problem-Statement:

A man has to cross a river using a boat. He carries three things with him, a sheep, a lion, a heap of grass. While crossing the river, he can take at most one thing with him on the boat at a time. **Problem is**, some of these objects are not to be left without surveillance (let's call these: incompatible).

We need to find an optimal way for the man to cross the river.

Mathematical Representation

The problem can be mathematically represented as states.

Let, any object on the Left bank has the state {1} and on the Right bank has state {0}. we have four objects, so initial state is {1,1,1,1} and goal state is {1,1,1,1}

Left Bank	River	Right Bank
state: 1 ----->	~~~~~ ----->	state: 0
:initial:		:Final:
{1, 1, 1, 1}		{0, 0, 0, 0}

So, we will have a graphical network of 2^4 nodes.

For simplicity and keeping the notations clean, let's use an upper case alphabet as label for each node state.

Each node has the form: {x, x, x, x} => {man, sheep, grass, lion}

So, we have the Relation:

```
from itertools import product

states = [list(s) for s in product([1, 0], repeat=4)]
num = 'A'
labels = []
nodes = {}
for i in range(16):
    nodes[num] = states[i]
    print(f'{states[i]} --> {num}')
    i+=1
    num = chr(ord(num)+1)
```

```
[1, 1, 1, 1] --> A
[1, 1, 1, 0] --> B
[1, 1, 0, 1] --> C
```

```

[1, 1, 0, 0] --> D
[1, 0, 1, 1] --> E
[1, 0, 1, 0] --> F
[1, 0, 0, 1] --> G
[1, 0, 0, 0] --> H
[0, 1, 1, 1] --> I
[0, 1, 1, 0] --> J
[0, 1, 0, 1] --> K
[0, 1, 0, 0] --> L
[0, 0, 1, 1] --> M
[0, 0, 1, 0] --> N
[0, 0, 0, 1] --> O
[0, 0, 0, 0] --> P

```

Constraints: We introduce constraints as defined by the problem

- $\{0, 1, 1, x\}$ OR $\{1, 0, 0, x\}$: Sheep and Grass should not be on the same bank without man
- $\{0, 1, x, 1\}$ OR $\{1, 0, x, 0\}$: Lion and Sheep should not be on the same bank without man

Definition: Traversing between nodes realistically translates to one movement of boat. $LeftBank \rightarrow RightBank \rightarrow LeftBank$

e.g. $A \rightarrow K$ basically means: Man and grass on the right, sheep and Lion on the left

Notice: that this traversal can't be continued further because $K = \{0, 1, 0, 1\}$: forbidden state.

To visualise the search algorithm, we can draw a **search tree**. To dive deeper into the problem, I'll be making the search tree for finding all possible solutions.

defining constraints in a bool function

```

def isAllowed(node):
    if node[1] == node[2] and node[0] != node[1]:
        return False
    elif node[1] == node[3] and node[0] != node[1]:
        return False
    else:
        return True

```

Defining the edges:

now that we have a boolean constraint checker `isAllowed`, we can formally define **valid transitions** between states.

A transition corresponds to **one boat move**:

- The man must always move.
- He may take **at most one** of {sheep, grass, lion} with him.
- The resulting state must satisfy the constraints.

We generate the edge set by iterating over all allowed states and simulating all physically possible boat moves.

```

state_map = {tuple(v): k for k, v in nodes.items()}
edges = []

for label, state in nodes.items():
    if not isAllowed(state):
        continue

    for item in [None, 1, 2, 3]:

        if item is not None and state[item] != state[0]:
            continue

```

```

next_state = state.copy()
next_state[0] = 1 - next_state[0]

if item is not None:
    next_state[item] = 1 - next_state[item]

edges.append((label, state_map[tuple(next_state)]))

```

This gives us the **complete transition relation** of the problem, upto forbidden states

Directed Graph Representation

using the above edge set, we now construct a **directed graph** where:

- Nodes represent states
- Edges represent valid boat crossings

```

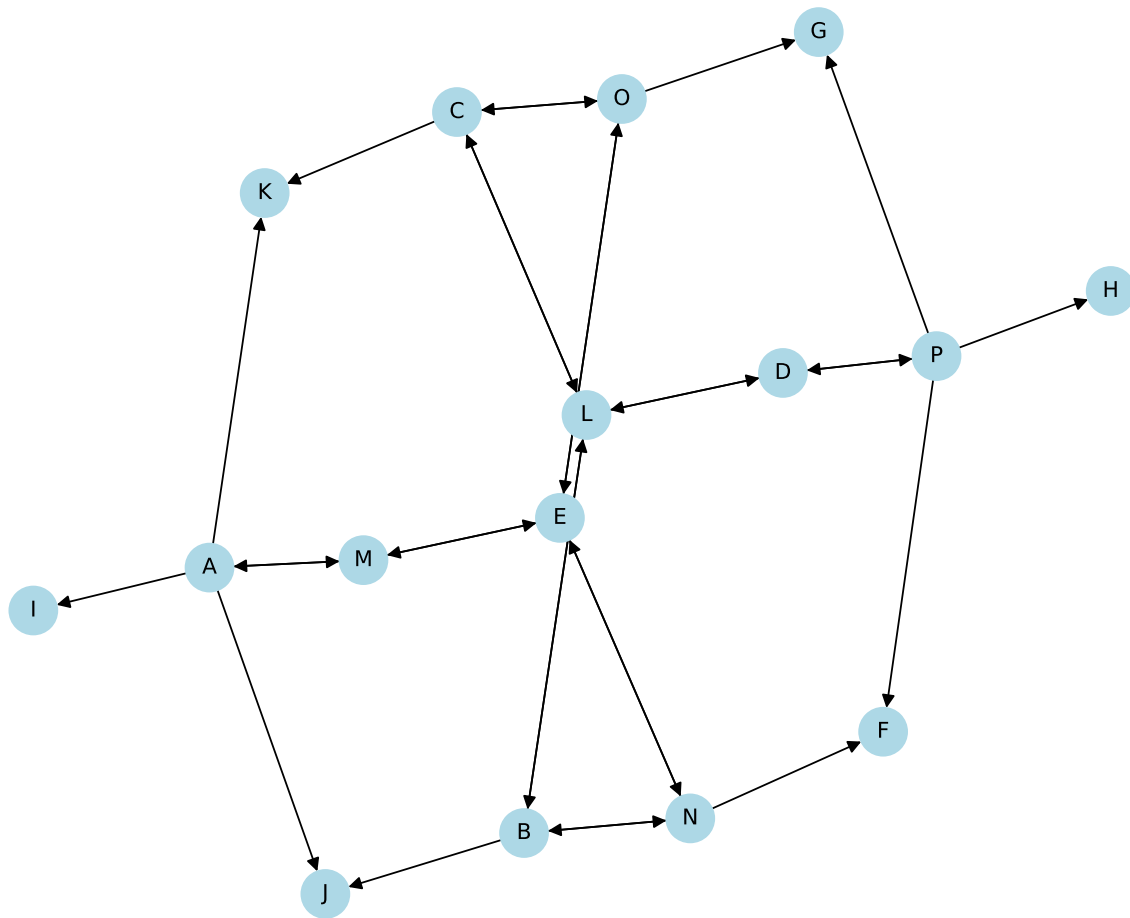
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph(edges)
pos = nx.nx_agraph.graphviz_layout(G)

plt.figure(figsize=(10, 8))
nx.draw(G, pos, with_labels=True, node_size=700, node_color="lightblue", arrows=True,
        ↪ arrowsize=15)
plt.title("State Transition Graph")
plt.show()

```

State Transition Graph



This graph encodes **all legal movements** in the system.

Generic Search Algorithm (Breadth-First Search)

We now apply **generic search** to find an optimal path from:

- **START** = **A** (initial state $\{1,1,1,1\}$)
- **TARGET** = **P** (goal state $\{0,0,0,0\}$)

We explicitly track:

- **LIST**: the frontier
- **VISIT**: visited nodes

The algorithm prints the updated **LIST** and **VISIT** at each step. `from collections import deque`

```
from collections import deque
```

```
START = "A"  
TARGET = "P"
```

```
LIST = deque([START])  
VISIT = []
```

```

parent = {START: None}

print(f"START = {START}, TARGET = {TARGET}\n")

while LIST:

    current = LIST.popleft()
    VISIT.append(current)

    print(f"Expanded: {current}")
    print(f"LIST : {list(LIST)}")
    print(f"VISIT : {VISIT}\n")

    if current == TARGET:
        print("Target Found!")
        break

    if 'G' in locals():
        for neighbor in G.successors(current):
            if neighbor not in VISIT and neighbor not in LIST:
                parent[neighbor] = current
                LIST.append(neighbor)

```

START = A, TARGET = P

Expanded: A
LIST : []
VISIT : ['A']

Expanded: I
LIST : ['M', 'K', 'J']
VISIT : ['A', 'I']

Expanded: M
LIST : ['K', 'J']
VISIT : ['A', 'I', 'M']

Expanded: K
LIST : ['J', 'E']
VISIT : ['A', 'I', 'M', 'K']

Expanded: J
LIST : ['E']
VISIT : ['A', 'I', 'M', 'K', 'J']

Expanded: E
LIST : []
VISIT : ['A', 'I', 'M', 'K', 'J', 'E']

Expanded: O
LIST : ['N']
VISIT : ['A', 'I', 'M', 'K', 'J', 'E', 'O']

Expanded: N
LIST : ['G', 'C']

```
VISIT : ['A', 'I', 'M', 'K', 'J', 'E', 'O', 'N']
```

```
Expanded: G
```

```
LIST  : ['C', 'F', 'B']
```

```
VISIT : ['A', 'I', 'M', 'K', 'J', 'E', 'O', 'N', 'G']
```

```
Expanded: C
```

```
LIST  : ['F', 'B']
```

```
VISIT : ['A', 'I', 'M', 'K', 'J', 'E', 'O', 'N', 'G', 'C']
```

```
Expanded: F
```

```
LIST  : ['B', 'L']
```

```
VISIT : ['A', 'I', 'M', 'K', 'J', 'E', 'O', 'N', 'G', 'C', 'F']
```

```
Expanded: B
```

```
LIST  : ['L']
```

```
VISIT : ['A', 'I', 'M', 'K', 'J', 'E', 'O', 'N', 'G', 'C', 'F', 'B']
```

```
Expanded: L
```

```
LIST  : []
```

```
VISIT : ['A', 'I', 'M', 'K', 'J', 'E', 'O', 'N', 'G', 'C', 'F', 'B', 'L']
```

```
Expanded: D
```

```
LIST  : []
```

```
VISIT : ['A', 'I', 'M', 'K', 'J', 'E', 'O', 'N', 'G', 'C', 'F', 'B', 'L', 'D']
```

```
Expanded: P
```

```
LIST  : []
```

```
VISIT : ['A', 'I', 'M', 'K', 'J', 'E', 'O', 'N', 'G', 'C', 'F', 'B', 'L', 'D', 'P']
```

```
Target Found!
```

Extracting the Solution Path

Once the target is reached, we backtrack using the `parent` map to get the path.

```
path = []
node = TARGET
while node:
    path.append(node)
    node = parent[node]

path.reverse()
print("Solution Path:", " -> ".join(path))
```

```
Solution Path: A -> M -> E -> O -> C -> L -> D -> P
```

Interpretation of the Solution

Each transition corresponds to a **single river crossing**.

Translated back to reality, the sequence implies:

1. Man takes sheep across
2. Man returns alone
3. Man takes lion across

4. Man brings sheep back
5. Man takes grass across
6. Man returns alone
7. Man takes sheep across

This is a valid solution under all the given constraints.

Full Search Tree Visualization

Finally, we construct the **entire search tree**, marking:

- **Forbidden states** in red
- **Solution path nodes** in green
- Remaining explored nodes in light blue

```
tree_edges = []
visited = set([START])
queue = deque([START])

while queue:
    u = queue.popleft()

    if u in G:
        for v in G.successors(u):
            if v not in visited:
                visited.add(v)
                tree_edges.append((u, v))
                queue.append(v)

T = nx.DiGraph(tree_edges)

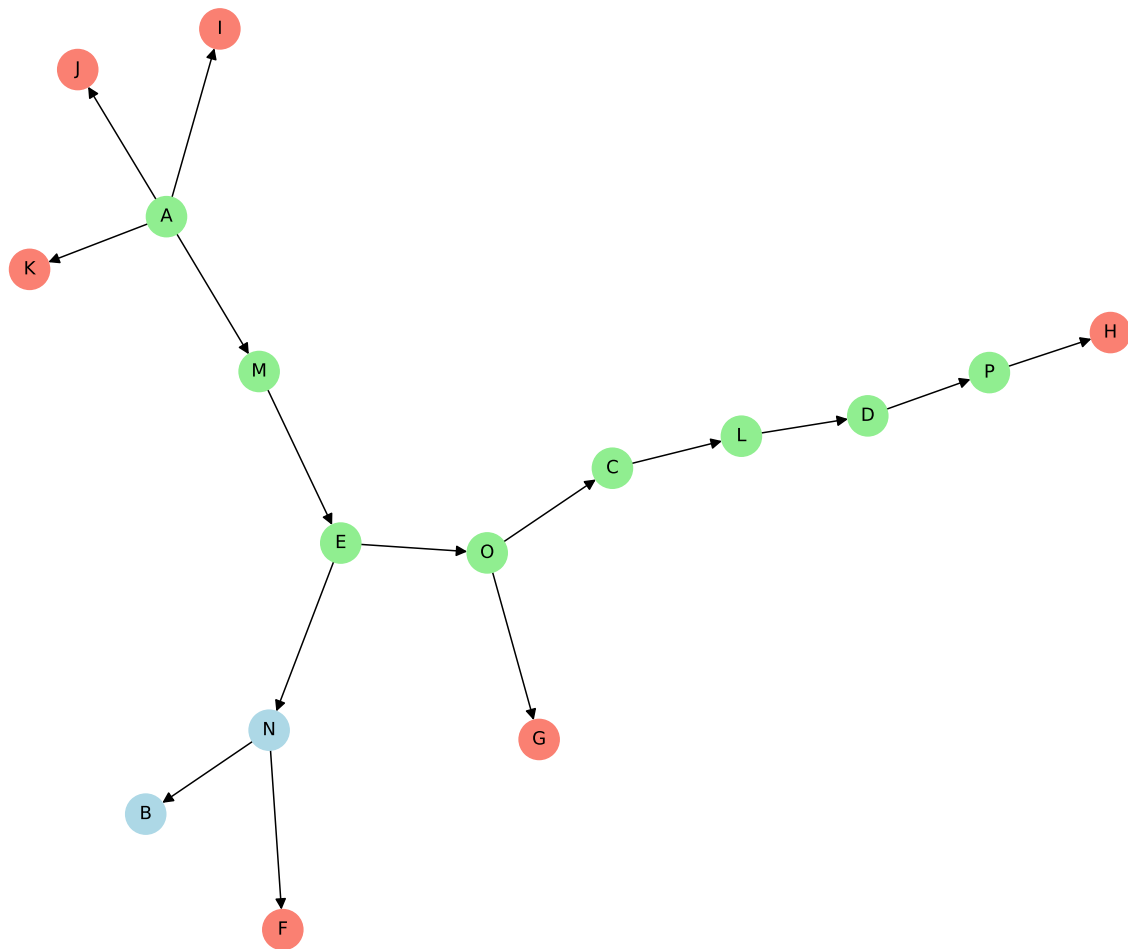
pos = nx.nx_agraph.graphviz_layout(T)
solution_nodes = set(path)

forbidden_nodes = [k for k, v in nodes.items() if not isAllowed(v)]

node_colors = []
for n in T.nodes():
    if n in solution_nodes:
        node_colors.append("lightgreen")
    elif n in forbidden_nodes:
        node_colors.append("salmon")
    else:
        node_colors.append("lightblue")

plt.figure(figsize=(12, 10))
nx.draw(T, pos, with_labels=True, node_color=node_colors, node_size=700, arrows=True,
        ↪ arrowsize=15)
plt.title("Complete Search Tree\nGreen = Solution Path, Red = Forbidden States")
plt.show()
```

Complete Search Tree
Green = Solution Path, Red = Forbidden States



My View of This Assignment

The main goal of this assignment for me has been to learn the intuition and mental model to approach search algorithms. Learning ways to represent the whole procedure in python has been a bonus.

For making this **html/pdf**, i have used **quarto** combined with **jupyter** for python code blocks and **latex/markdown** for stylized text. As mentioned in teh disclaimer at the top, nothing in this document has been written by any sort of LLM or AI assistant. But of course, I have taken help of such in understanding the syntax and debugging the issues faced throughout the assignment (mostly related to python syntax or latex).

References used:

These references where mostly used to understand and visualise *state spaces*.

- **Artificial Intelligence**, by *Charu C. Aggarwal*
- **AI using Python**, *GCS Youtube* [link](#)